

SIMATS ENGINEERING

CO4 ASSESSMENT TOOL 2

NAME : JAIVANT.S

REG NO : 192311326

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE : CSA0613

Q13. Smart Warehouse Storage Optimization (Bin Packing)

Objective

To efficiently allocate packages of different sizes into storage units with fixed capacity using a greedy Bin Packing algorithm and minimize the number of storage units required.

Problem Description

In a smart warehouse, packages of different sizes must be stored efficiently in storage units with limited capacity. This problem can be modeled as a **Bin Packing Problem**, where packages represent items and storage units represent bins.

The main objective is to place all packages without exceeding the capacity of any storage unit while minimizing the total number of storage units used.

Constraints

The important constraints are:

- Each package must be placed in exactly one storage unit.
- The total size of packages in a storage unit cannot exceed its capacity.
- Package size should not be greater than bin capacity.
- Storage space utilization should be maximized.
- The number of storage units should be minimized.

Algorithm Used: First-Fit

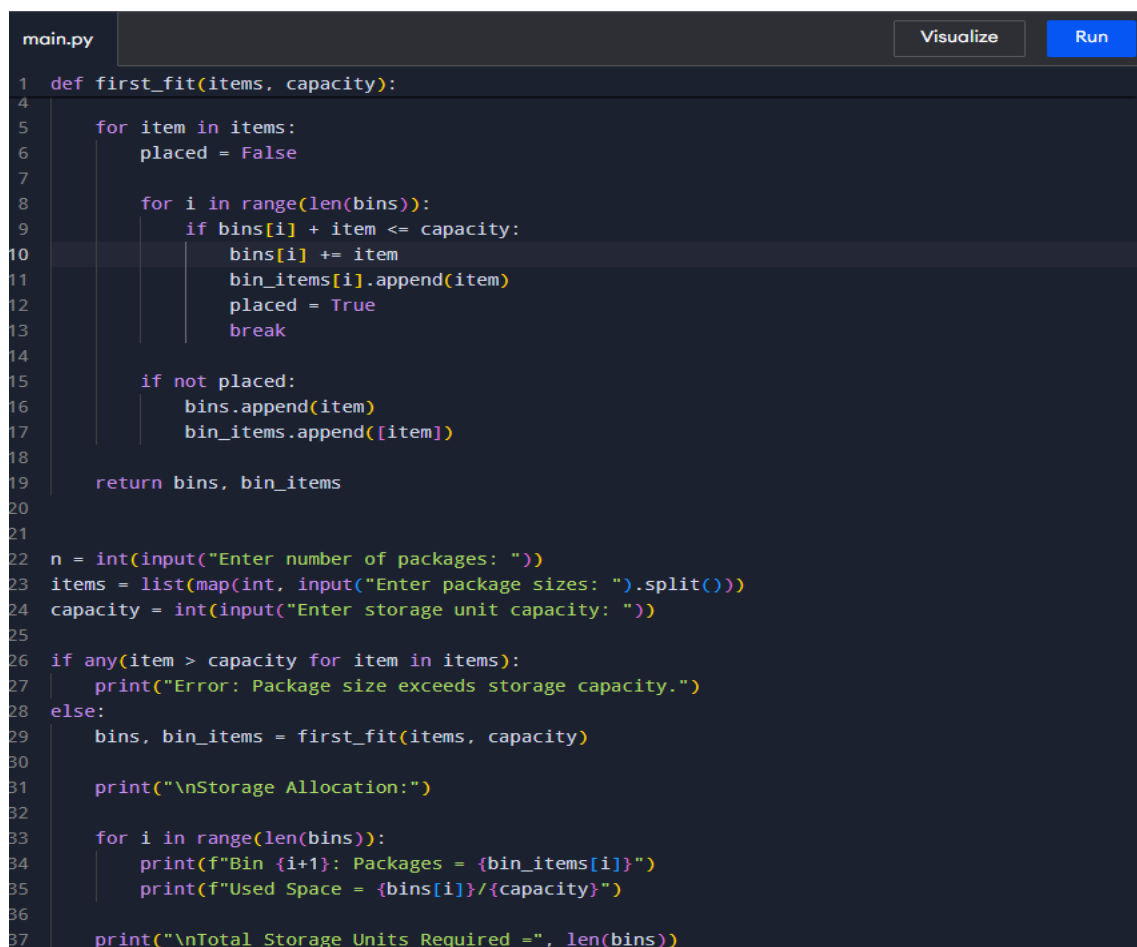
The **First-Fit Greedy Algorithm** places each package into the first storage unit that has enough available space.

If no existing storage unit can accommodate the package, a new storage unit is created.

Steps

1. Read the number of packages and their sizes.
2. Read the storage unit capacity.
3. Start with an empty list of storage units.
4. For each package, check existing storage units.
5. Place it in the first unit where it fits.
6. If it does not fit anywhere, create a new storage unit.
7. Display the package allocation and total number of storage units.

Source code:



```
main.py Visualize Run
1 def first_fit(items, capacity):
2
3     for item in items:
4         placed = False
5
6         for i in range(len(bins)):
7             if bins[i] + item <= capacity:
8                 bins[i] += item
9                 bin_items[i].append(item)
10                placed = True
11                break
12
13         if not placed:
14             bins.append(item)
15             bin_items.append([item])
16
17     return bins, bin_items
18
19
20
21
22 n = int(input("Enter number of packages: "))
23 items = list(map(int, input("Enter package sizes: ").split()))
24 capacity = int(input("Enter storage unit capacity: "))
25
26 if any(item > capacity for item in items):
27     print("Error: Package size exceeds storage capacity.")
28 else:
29     bins, bin_items = first_fit(items, capacity)
30
31     print("\nStorage Allocation:")
32
33     for i in range(len(bins)):
34         print(f"Bin {i+1}: Packages = {bin_items[i]}")
35         print(f"Used Space = {bins[i]}/{capacity}")
36
37     print("\nTotal Storage Units Required =", len(bins))
```

Output:

```
Output

Enter number of packages: 6
Enter package sizes: 4 8 1 4 2 1
Enter storage unit capacity: 10

Storage Allocation:
Bin 1: Packages = [4, 1, 4, 1]
Used Space = 10/10
Bin 2: Packages = [8, 2]
Used Space = 10/10

Total Storage Units Required = 2

=== Code Execution Successful ===
```

Analysis

The First-Fit algorithm attempts to use existing storage space before creating a new storage unit. For the given example, all six packages are efficiently allocated into two bins.

$$\text{Bin 1} = 4 + 1 + 4 + 1 = 10$$

$$\text{Bin 2} = 8 + 2 = 10$$

This results in **100% space utilization** for both storage units.

Trade-Off

The First-Fit algorithm is fast and easy to implement, but it may not always produce the minimum possible number of bins. More advanced algorithms can provide better optimization but require additional computation time.

Therefore, First-Fit provides a good balance between:

- Computational efficiency

- Storage utilization
- Fast package allocation

Time Complexity

In the worst case, each package may need to check all existing bins.

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Performance in Large Warehouses

For large-scale warehouses, greedy algorithms are useful because they can quickly allocate incoming packages without performing expensive calculations. This makes them suitable for real-time logistics operations.

The approach can be applied to:

- Warehouse storage allocation
- Truck loading
- Shipping containers
- Inventory management
- E-commerce logistics

Conclusion

The Smart Warehouse Storage Optimization problem was solved using the First-Fit Bin Packing algorithm. The algorithm efficiently allocates packages into storage units while respecting capacity constraints. Although it may not always produce the optimal solution, its simplicity and speed make it suitable for practical warehouse and logistics applications.